

A new rule builds OPDI's flight tracks

Version 2 — the release note: what changed in the pipeline, what it recovers, and why every track_id you hold stops joining

EUROCONTROL Performance Review Unit

2026-09-03

Summary

OPDI now builds track_id with a different rule, and no identifier from any earlier release joins to any identifier in this one.

Segmentation is the pipeline's decision about which state vectors belong to the same flight. It is made once, in step 02, and everything downstream inherits it: the flight list groups by `track_id`, events fire per track, measurements accumulate from a track's first sample. Every OPDI release up to v0.0.2 made that decision by grouping on (`icao24`, `callsign`) and cutting on time gaps. From this release forward the pipeline groups on `icao24` alone and cuts when the airframe's callsign genuinely changes. The configuration key is `segmentation.method`, and its default moved from "legacy" to "standard".

What it recovers. On a full day of state vectors through the production steps, the share of Network Manager flights that come out as exactly one track carrying nothing else rises from 51.98% to 91.96% on 2025-06-05, and from 53.81% to 92.87% on 2024-06-05. Score the turnaround as part of the flight — block to block, so stand and taxi samples count — and the same change is 34.91% to 90.88%, a gain of +55.97 points.

Why this page exists. Version 1 of this study measured eight candidate rules in a harness that read a pre-built track table, and recommended this one. A harness is not the pipeline, and the question a recommendation cannot answer is whether its result survives the real thing. It does. The pipeline and the harness disagree by at most 1.08 points across two periods and two scoring intervals — four independent comparisons of the same quantity, agreeing to about a point.

What it costs. The flight list names slightly fewer arrival aerodromes: 67.15% of ground-truth flights against 67.81%, at essentially unchanged accuracy. Across both ends the flight list's own score falls 0.61% on 2025 and 0.97% on 2024. The mechanism is merging, and [the payoff section](#) measures it.

What breaks. Every `track_id` changes. The two rules hash different group keys and use different suffixes, so the count of identifiers surviving the release is zero for reasons visible in the code rather than in any measurement. The partition itself shrinks by 31.77% on 2025 and 30.70% on 2024. A consumer joining on `track_id` across the release boundary gets an empty result, not an error. **Nothing in the published data says which rule produced a row** — `osn_tracks` carries no version column, so the release date is the only discriminator. That is [what breaks](#), and it is the section to read if you read only one.

i How to read this page

This is a release note, not a study. It reports one pipeline run per period, three segmentation arms each, through the production steps `01 → 02 → 02a → 03`. [What changed](#) is the consumer-visible list. [How a segmentation is scored](#) defines every term used later, so this page stands alone and does not send you to V1 for a definition. [What it improves](#) and [which change does the work](#) are the measurement. [What it costs](#) is downstream. [What breaks](#) and [what ships](#) are the operational sections.

Every number here is an expression evaluated at render time over a committed CSV — `data/pipeline_2025.csv` and `data/pipeline_2024.csv` for the pipeline run, and V1's own `arms_*.csv` for the harness figures the cross-validation compares against. Both pipeline files are declared by `benchmarks/regenerate_track_v2.py`, which this page runs before drawing anything, which invokes `benchmarks/track_pipeline_v2.py` to produce them, and which stamps each output with the code that produced it in `data/_manifest.json` — script, arguments, git SHA, and a fingerprint over the source files the job depends on. Nothing is typed into the prose, which is what stops the text and the tables drifting apart. Two exceptions are named where they appear: the configuration defaults, which are code rather than measurement, and one diagnostic quoted from `config.py`.

V1's figures are a different measurement of the same quantity, not a baseline these are differences from. V1 scored a harness over three days; V2 scores the pipeline over one. Where both are quoted, they are quoted as two answers to one question.

What changed

What	Was	Is now
<code>segmentation.method</code>	"legacy"	"standard"
<code>FLIGHT_LIST_VERSION</code>	v2.0.0 / v3.0.0	v5.0.0
<code>FLT_ID</code> (flight list)	lexicographically smallest callsign in the track	the track's dominant non-blank callsign
<code>FLT_ID</code> on unlabelled overflights	NULL	""

What	Was	Is now
Airport events (step 04)	one event per callsign broadcast during a zone crossing	one event per zone crossing

The two rules, stated in full. `legacy` groups samples by `SHA2(icao24 || callsign)`, orders them by time, and starts a new track when the gap to the previous sample exceeds 30 minutes, or exceeds 15 minutes while the aircraft is below 5,000 ft. The identifier it stamps carries a `_{year}_{month}` suffix. `standard` groups by `SHA2(icao24)`, keeps both gap rules unchanged, and adds one break: the last *non-blank* callsign seen on this airframe differs from the current one. Its identifier carries a sample offset instead of a month suffix.

The break compares the last non-blank callsign rather than the raw column because ADS-B broadcasts the callsign in its own message type and it is blank until the crew sets it. Comparing raw values would fire on every blank sample and reproduce exactly the fragmentation the change removes. The lookback is bounded by the gap threshold, so it cannot reach back across a gap break into the previous flight.

Three changes reach data consumers directly, and none of them raises an error.

`FLT_ID` is now the dominant callsign of the track rather than the lexicographically smallest, because a blank sorts first and a `standard` track can carry a blank. Under legacy segmentation the two aggregates agree on every track — callsign is in the group key, so a legacy track carries one value by construction — so no historical data is affected.

Unlabelled overflights now carry "" where they carried NULL. The column previously used "" for unlabelled detected flights and NULL for unlabelled overflights: two spellings of “no callsign” in one column, so `WHERE FLT_ID IS NULL` and `WHERE FLT_ID = ''` answered the same question differently. **A consumer filtering overflights on IS NULL now silently gets nothing back.**

Step 04 emits one airport event per zone crossing. It previously grouped on `flight_id` without aggregating it, so an aircraft that broadcast two callsigns while crossing one runway emitted two entry-runway events for one crossing. On legacy tracks that could not happen, and the two behaviours are identical on every published release.

`osn_tracks` gains no version column. A row does not say which segmentation produced it. The release date is the only discriminator, which is why [what breaks](#) is written the way it is.

What has already happened to a sample

Side information: if you know the pipeline steps, skip to [the metrics](#).

Segmentation is step 02 of a chain, and the samples reaching it are not raw OpenSky rows. Step 01 restricts them to the OPDI bounding box and thins them to one sample per aircraft per 5 s interval, so every gap threshold in this study is measured against a 5 s grid.

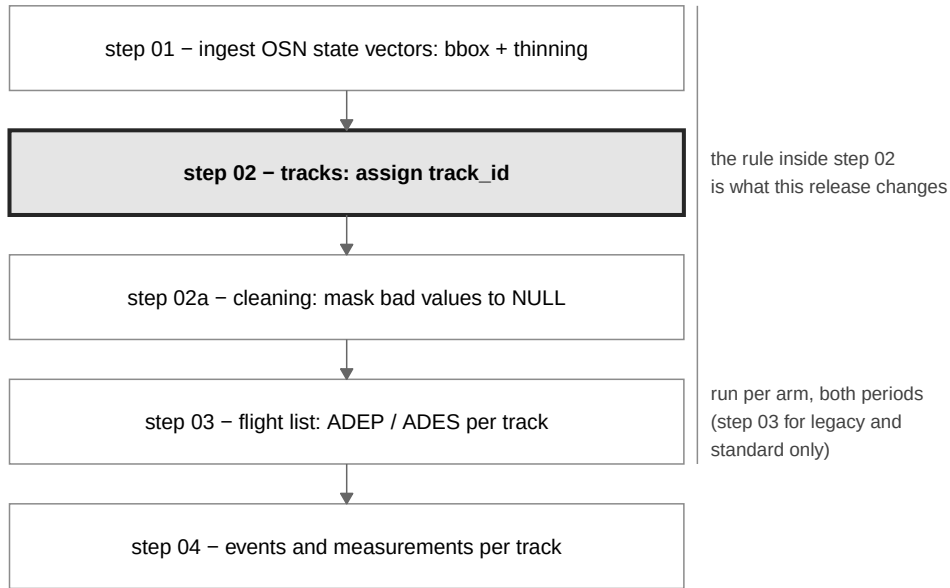


Figure 1: The production chain. Step 02 is where the rule changed; every step below it inherits the result, and this release ran the whole chain per arm rather than a harness over step 02’s output.

Step	What it does	Writes
01	Downloads OpenSky state vectors, restricts them to the OPDI bounding box, and thins them to one sample per aircraft per interval.	<code>osn_statevectors</code>
02	Groups samples into tracks and stamps <code>track_id</code> . This is the step that changed.	<code>osn_tracks</code>
02a	Masks values that cannot be right — duplicates, out-of-range positions, stale broadcasts, derivative spikes, isolated points — to NULL, keeping the row. Writes a second table rather than editing the first.	<code>osn_tracks_clean</code>
03	One flight per <code>track_id</code> : departure and destination aerodrome, and the flight’s callsign label.	<code>opdi_flight_list</code>

Step	What it does	Writes
04	Milestones per track — take-off, top-of-climb, level segments, landing — and the measurements attached to them.	events, measurements

Each arm ran $01 \rightarrow 02 \rightarrow 02a$ for its period, and **legacy** and **standard** ran 03 as well. The three arms of a period share one ingested copy of the day, so a difference between arms cannot be an artefact of having ingested the day three times.

How a segmentation is scored

Nothing after this section introduces a metric.

Ground truth is Network Manager’s flight table, joined to ADS-B on **icao24** — the 24-bit airframe address, which Network Manager carries as **AIRCRAFT_ADDRESS**. A ground-truth flight is an airframe and a span of time; a state vector is attributed to the flight of the same airframe whose span holds its timestamp. A track and a flight are *matched* when at least one of the track’s samples is attributed to that flight. The join is never on callsign, because callsign is exactly what the rules under comparison disagree about.

Every ground-truth flight falls in one of three classes, which sum to 100%.

- **clean** — the flight’s samples are in one track, and that track carries no other flight. This is **clean_match_pct**, the headline.
- **fragmented** — the flight’s samples span several tracks, none of which carries another flight.
- **merged** — the flight’s dominant track also carries at least one other flight.

A flight that is both counts as **merged**, because merging is the worse failure: a merged flight is unrecoverable downstream, where a fragmented one is at least present in pieces. One stray sample in a second track costs a flight its clean match, so **clean_match_pct** is a lower bound and is read as one.

Two entropy-based clustering measures name the two failure modes on a 0–1 scale. **Homogeneity** asks whether each track holds samples from only one flight, so it falls with merging. **Completeness** asks whether each flight’s samples all landed in one track, so it falls with fragmentation. Both are weighted over every sample in the period, and most samples sit in tracks that every arm gets right, so they span a narrow range and name the failure modes rather than rank the arms. The three-class rate is what ranks them.

Each arm is scored twice, over two intervals of the same partition.

- The **airborne** interval runs from take-off to landing, $[t_{\text{off}}, t_{\text{land}}]$. A sample taken on stand or on a taxiway falls outside it and is attributed to no flight.
- The **gate-to-gate** interval runs off-block to in-block, so the turnaround counts. This matters because an OPDI track is supposed to carry it: first- and last-seen events fire on it, taxi phases are extracted from it, and every cumulative measurement starts at the track’s first sample. A rule that shatters a flight between the gate and the runway threshold scores a clean airborne match and hands over a broken product.

The gate interval is where a rule carrying callsign in its group key comes apart, because that is when the callsign is blank or rotating: the aircraft is on stand, the crew has not set the next leg’s callsign, and every blank sample forms a group — and therefore a track — of its own.

Block times come from APDF where a movement was measured. The departure side falls back to Network Manager’s AOBT_3 and so exists for substantially every flight; the arrival side has no fallback, because ARVT_3 is a landing time and not a gate arrival, and is therefore APDF-only. Where a block time is missing the interval is extended past the airborne endpoint by the median measured taxi on that side, over the flights of that period that measured one.

The two intervals score the same flights here. The gate interval can in principle admit flights the airborne one does not — a departure seen only on the taxiway — so the denominators are reported rather than assumed. On 2025 they differ by 3 flights out of 29,160, and on 2024 by 0. Nothing below is a change of denominator wearing the clothes of a change of rate.

The sample. One day per period, run whole: 48,055,850 state vectors on 2025-06-05 and 46,204,353 on 2024-06-05. 32,404 Network Manager flights are in scope for 2025 and 30,407 for 2024, of which 29,160 and 27,409 respectively have at least one ADS-B sample inside their airborne interval and are what the rates are computed over.

What it improves

Table 1: Airborne (columns 3–5) and gate-to-gate (columns 6–8). Each triple sums to 100%.

Period	Arm	clean	fragmented	merged	clean	fragmented	merged
2025	legacy	51.98%	47.47%	0.55%	34.91%	64.54%	0.55%
2025	airframe_only	89.71%	5.67%	4.63%	89.68%	5.73%	4.60%
2025	standard (ships)	91.96%	7.17%	0.87%	90.88%	7.83%	1.30%
2024	legacy	53.81%	45.80%	0.39%	34.69%	64.91%	0.40%
2024	airframe_only	90.79%	5.09%	4.12%	90.71%	5.17%	4.12%
2024	standard (ships)	92.87%	6.45%	0.67%	91.78%	7.10%	1.12%

legacy fragments 47.47% of flights on 2025 and merges 0.55%. Almost all of its failure is the splitting and almost none is the opposite error. **standard** fragments 7.17% and merges 0.87%: a fragmentation fix that pays for itself several times over in a small amount of new merging.

Scored gate to gate, **legacy** falls to 34.91% while **standard** holds 90.88%. The wider interval costs **standard** 1.08 points on 2025 and **legacy** 17.07. Adding the turnaround adds samples, and added samples can only fragment a partition further; the size of the two penalties is the difference between a rule that carries callsign in its group key and one that does not.

Table 2: Homogeneity falls with merging, completeness with fragmentation. **matched tracks** counts tracks carrying at least one ground-truth sample; **tracks written** counts every track step 02 wrote for the day.

Period	Arm	homogeneity	completeness	matched tracks	tracks written
2025	legacy	0.9998	0.9941	46,577	81,531
2025	airframe_only	0.9974	0.9962	30,233	45,914
2025	standard (ships)	0.9997	0.9951	31,778	50,213
2024	legacy	0.9999	0.9945	42,704	79,142
2024	airframe_only	0.9977	0.9963	28,406	44,955
2024	standard (ships)	0.9997	0.9955	29,593	47,322

Completeness rises from 0.9941 to 0.9951 on 2025 while homogeneity falls from 0.9998 to 0.9997: less fragmentation, marginally more merging, the same finding the three-class rates give and the reason both are reported.

Running **standard** costs no more cluster time than running **legacy**. Arm runtimes across the six runs span 68.2 to 90.2 minutes, and which arm is slowest reverses between the two periods — on 2025 **legacy** took 90.2 minutes against **standard**’s 68.2, and on 2024 the order is the other way round. These are wall-clock times on a shared cluster and settle nothing finer than that.

Does the harness result survive the pipeline?

This is the question the release was made to answer, and it is why the arms were re-run through the production steps rather than scored again in the harness that recommended them.

V1 measured segmentation by applying each candidate rule to an already-built, already-cleaned track table. That design bought eight rules for the price of one pipeline run, and it left one thing open: a re-implementation of a rule scored over a re-used table is not the rule the pipeline runs over the table the pipeline builds. Cleaning runs *after* segmentation in production and *before* it in the harness. If the recommendation depended on that ordering, or on any other difference between the two, the recommendation was an artefact.

Table 3: **standard** minus **legacy**, in points of clean matching. Two independent measurements of one quantity over different samples: V1 scored three days in a harness, V2 scores one day through the production steps.

Period	Interval	pipeline (V2)	harness (V1)	difference
2025	airborne	+39.98	+40.65	-0.67
2025	gate-to-gate	+55.97	+55.87	+0.10
2024	airborne	+39.06	+38.96	+0.10
2024	gate-to-gate	+57.08	+56.00	+1.08

The two measurements agree to within 1.08 points on all four comparisons. The pipeline run confirms the harness result: the gain V1 recommended on is the gain production delivers, and the harness’s shortcuts did not manufacture it.

The agreement also disciplines the claim. Four numbers this close, from different samples and different code paths, say that the effect is a property of the segmentation rule rather than of the day, the ordering of cleaning, or the implementation.

Which change is doing the work

standard is two changes to **legacy**, and either could have been carrying the result. Dropping callsign from the group key stops blank callsigns from forming tracks. Adding the callsign-change break puts back the separation the group key used to supply. The middle arm, **airframe_only**, is the first change without the second, and it exists so the gain can be split.

Table 4: The gain split at its midpoint, airborne interval. The two steps sum to the total by construction.

Period	Step	What it changes	clean matching (points)	merged (points)
2025	legacy → airframe_only	drop callsign from the group key	+37.73	+4.07
2025	airframe_only → standard	add the callsign-change break	+2.25	-3.76
2025	total		+39.98	+0.32
2024	legacy → airframe_only	drop callsign from the group key	+36.98	+3.73
2024	airframe_only → standard	add the callsign-change break	+2.08	-3.44
2024	total		+39.06	+0.29

Dropping callsign from the group key is nearly all of it: 37.73 of the 39.98 points on 2025, and 36.98 of 39.06 on 2024. The mechanism is blunt. With callsign in the key, a sample decoded without one forms its own group and therefore its own track; a diagnostic taken while the rules were implemented and recorded in `config.py` — not read from a CSV here — puts 42.4% of legacy’s tracks in that state. Fragmentation is what that produces, and it is an artefact of the key rather than anything about the flight.

The callsign-change break earns the rest by removing merging. `airframe_only` alone merges 4.63% of flights on 2025 against legacy’s 0.55%: grouping on the airframe alone joins two consecutive legs of the same aircraft into one track, and the second leg stops existing downstream. Restoring the break cuts that to 0.87% and adds +2.25 points of clean matching on 2025 and +2.08 on 2024. The trade is visible in the fragmentation column too — **standard** fragments 1.51 points more than `airframe_only`, the cost of splitting on a signal that is sometimes noise — and it is worth paying because merging is the failure that loses a flight outright.

Both halves earn their place on both periods, which is why the shipped rule is the combination and not either piece.

What it costs downstream

Step 03 — the expensive step — ran for `legacy` and `standard` only. `airframe_only` is the segmentation ablation’s midpoint and has no downstream question of its own: what a consumer needs is what the release changes, which is the before and the after. **Its ADEP/ADES cells are blank because no run produced them, not because a run failed.**

Table 5: Coverage is the share of ground-truth flights given an aerodrome at all; accuracy is the share of those that are right. **score** is correct minus twice wrong, summed over both ends — the flight list’s own trade-off between naming an aerodrome and naming it wrong.

Period	Arm	ADEP coverage	ADEP accuracy	ADES coverage	ADES accuracy	score
2025	legacy	80.25%	97.36%	67.81%	98.30%	44,795
2025	airframe_only	—	—	—	—	—
2025	standard (ships)	80.20%	97.36%	67.15%	98.21%	44,520
2024	legacy	79.57%	97.16%	67.39%	97.98%	41,389
2024	airframe_only	—	—	—	—	—
2024	standard (ships)	79.64%	96.96%	66.69%	97.86%	40,986

Accuracy is unchanged and coverage falls slightly. ADEP accuracy moves +0.00 points on 2025 and -0.20 on 2024. The loss is on the arrival side: ADES coverage falls 0.66 points on 2025 and 0.70 on 2024. Over both ends the score falls 275 points on 2025 and 403 on 2024, which is 0.61% and 0.97% of the total.

The mechanism is merging, the only failure mode the new rule makes worse. A merged track holds two legs and gets one arrival aerodrome named for it, so the second leg's arrival is not named at all — a coverage loss rather than an accuracy loss, which is what the table shows. **standard** merges 0.87% of flights on 2025 against **legacy**'s 0.55%, and that difference is where the arrival coverage went.

Why the downstream effect is so much smaller than the segmentation gain. Read the denominator. Of 32,404 ground-truth flights on 2025, **legacy** already names an ADEP for 26,005 of them and **standard** for 25,989. The flight list resolves an aerodrome from the samples near the ground at either end of a track, and a fragmented flight usually keeps those samples in *some* fragment — the piece holding the departure roll still sees the departure aerodrome. Segmentation moves the flights where that is not true, a small share of a large denominator. A +39.98-point gain in clean matching beside a 0.61% change in aerodrome score is not a null result; it is two metrics measuring different things, one of which was already close to its ceiling.

The gain is real for anything that consumes a track rather than a flight-list row. Events, taxi phases and every cumulative measurement are computed per track and are wrong in exactly the way segmentation was wrong.

What breaks

! A `track_id` is not a flight

It is the identifier of a partition, and this release repartitions. Every `track_id` published before it and every `track_id` published after it are drawn from disjoint sets. A join across the boundary returns an empty result rather than an error, which reads as missing data rather than as a broken key — and nothing in the data will tell you, because `osn_tracks` carries no column saying which rule built a row.

No identifier survives, and no run measured that. It is true by construction, from two facts in `segmentation/methods.py`. **legacy** hashes `icao24 || callsign` and suffixes `_{year}_{month}`; **standard** hashes `icao24` alone and suffixes a sample offset. Different hash inputs and different suffix shapes mean the count of identifiers common to the two releases is zero, and no measurement could return anything else.

The partition change was measured.

Table 6: Tracks carrying at least one ground-truth sample. The last column is the same comparison over every track step 02 wrote, whose denominator includes tracks no ground-truth flight touches.

Period	legacy tracks	standard tracks	change	same change, V1 harness	change in tracks written
2025	46,577	31,778	- 31.77%	-32.64%	-38.41%
2024	42,704	29,593	- 30.70%	-30.86%	-40.21%

The number of tracks falls 31.77% on 2025 and 30.70% on 2024, and V1's harness measured the same reduction independently at 32.64% and 30.86%. Both arms see the same airframes, so tracks-per-airframe moves by the same ratio: a consumer counting tracks per aircraft per day sees that proportional fall, and none of the tracks it loses are lost flights.

What a consumer has to do. Re-key anything that joins on `track_id` across the boundary, and treat the release date as the discriminator between the two regimes, because the data carries no other. Re-processing a past month with `segmentation.method = "legacy"` reproduces that month's identifiers; running the default over it does not.

 The cleaned table cannot reproduce its own identifiers

`track_id` is assigned in step 02 from **raw** altitudes. Step 02a then masks the values that fail its filters to NULL, including 21.09% of `baro_altitude` on 2025 and 20.94% on 2024. Legacy's low-altitude split tests `altitude < 5,000 ft`, and a NULL compared against 5,000 is neither true nor false, so re-deriving the partition from `osn_tracks_clean` misses splits the original run made. Anyone doing so gets a different answer and concludes the segmentation is non-deterministic. It is not. The input it was computed from is no longer in the table being read.

Reproducing a published `track_id` needs `segmentation.method = "legacy"` **and** the raw altitudes, which means re-ingesting rather than reading the cleaned table.

What ships

Setting	Ships as	What it means
<code>segmentation.method = "standard"</code>	Get up on icao24; break on a gap, or on a genuine non-blank call sign change. "legacy" stays selectable and reproduces published identifiers.	

Setting	Ships as	What it means
<code>segmentation.gap_30minutes</code>	Split	Split when the gap to the previous sample exceeds this, at any altitude.
<code>segmentation.low_alt_gap_splitminutes</code>	Split	this shorter gap when the aircraft is low.
<code>segmentation.low_5000ft</code>		The altitude below which the shorter gap applies.
<code>segmentation.callsign_lookbackminutes</code>	None	Back the callsign comparison looks. <code>None</code> follows <code>gap_minutes</code> , so the lookback cannot reach across a gap break into the previous flight.

The three gap thresholds are unchanged from `legacy`. V1 swept them over 235 combinations and found the shipped cell within a point of the best cell anywhere, with the ordering of the one axis worth moving reversing between the two periods — so nothing was retuned, and this release changes the group key and the break condition only.

Re-running a past month with the defaults will not reproduce it. It will produce the same aerodromes for substantially the same flights, and different identifiers for all of them.

Limitations

One day per period. Every rate on this page is 2025-06-05 or 2024-06-05, run whole through the production steps. Two independent days agreeing, and agreeing with V1’s three-day harness sample, is the whole of the evidence that the result is not a property of a day.

Five of V1’s eight rules were not re-run. The pipeline arms are `legacy`, `airframe_only` and `standard`. The ground-anchored, airport-anchored, vertical-profile, month-suffix and traffic-style rules exist only as harness measurements, and nothing here says how they would behave in production.

Step 04 was not measured. Events and measurements inherit the new partition, and no arm here scored them. The airport-event change described in [what changed](#) is a code change with a stated mechanism, not a measured delta.

Ground truth is Network Manager flights only. A track that merges a Network Manager flight with a military or general-aviation flight the ground truth does not carry is scored as clean, so the merge rates here are lower bounds.

V1’s figures are a second measurement, not a baseline. Where the cross-validation sets V1 beside V2 it is comparing two answers to one question over different samples. Neither column is the other’s error bar.

Published data carries no marker of which segmentation produced it. Distinguishing the two regimes depends on knowing the release date. This is stated three times on this page because it is the failure a consumer cannot detect from the data.

The release's safety guards are keyed on configuration, because the data carries no provenance. Both `flights.py` and `events.py` skip callsign resolution when the run stamps a legacy version string, so a correctly configured legacy re-run reproduces its released month byte for byte. Neither guard can ask the question it actually needs answered — *were the tracks I am reading built with the legacy rule?* — because no column records it. A legacy-stamped run over an `osn_tracks` rebuilt with `standard` therefore fans out while stamping the frozen version, and nothing in the output reveals it. `flights.py`'s `tracks_table == "osn_tracks"` clause distinguishes raw tracks from *clean* ones, not legacy-built from standard-built. This is the cost of shipping without a segmentation marker, and it already affects one shipped configuration: `benchmarks/event_bench.py`'s rungs L00–L12 inherit `events_v0.0.2` and so run unresolved.

Production and the benchmark resolve the callsign over different rows at a month boundary. Production resolves inside the reader, *after* the month filter, because moving it earlier defeats partition pruning. The benchmark resolves over its whole redirected table. The two therefore agree for every track that lies inside one month and can disagree for one that straddles a boundary. No track in either sample straddles one — V1's payoff runs three June days, V2 runs 2025-06-05 and 2024-06-05 — so nothing on this page is affected. A whole-month comparison would meet it, which is why it is written down rather than left for someone to rediscover.

Provenance note

`pipeline_2024.csv` and `pipeline_2025.csv` were produced at opdi commit 2f95ec4. Two commits landed afterward that touch a declared dependency of both files. e88516a edits `benchmarks/track_pipeline_v2.py` to pass `args.period` instead of the `PERIODS` dict into `extents_name`, fixing the filename of the extents export — an output that same commit stops declaring. 090db8a adds three paragraphs to a docstring in `benchmarks/track_score.py` and changes no executable line. Neither touches the code that computes `n_tracks`, `clean_match_pct`, or any other value these two CSVs carry.

`benchmarks/provenance.py` fingerprints a declared dependency by hashing its full file bytes, so it cannot tell a filename argument or a docstring from a change to the arithmetic. Both commits mark `pipeline_2024.csv` and `pipeline_2025.csv` stale, and `--check` reports them that way. Reproducing them to clear the flag costs about 3.8 hours per period, 7.6 hours total, and would write files identical to the ones committed here, because nothing that ran actually changed. This page renders with `OPDI_RENDER=skip` and reports these two outputs as staged rather than paying that cost for an identical result.